```
            std::this_thread::yield();
        }
    }
```

This implementation of `run_pending_task()` is lifted straight out of the main loop of the `worker_thread()` function, which can now be modified to call the extracted `run_pending_task()`. This tries to take a task of the queue and run it if there is one; otherwise, it yields to allow the OS to reschedule the thread. The Quicksort implementation next is a lot simpler than the corresponding version from listing 8.1, because all the thread-management logic has been moved to the thread pool.

Listing 9.5. A thread pool–based implementation of Quicksort

```
template<typename T>
struct sorter                    <-(1)
{
    thread_pool pool;            <-(2)

    std::list<T> do_sort(std::list<T>& chunk_data)
    {
        if(chunk_data.empty())
        {
            return chunk_data;
        }

        std::list<T> result;
        result.splice(result.begin(),chunk_data,chunk_data.begin());
        T const& partition_val=*result.begin();
        typename std::list<T>::iterator divide_point=
            std::partition(chunk_data.begin(),chunk_data.end(),
                           [&](T const& val){return val<partition_val;});

        std::list<T> new_lower_chunk;
        new_lower_chunk.splice(new_lower_chunk.end(),
                               chunk_data,chunk_data.begin(),
                               divide_point);
```